

Visualización de BFS y DFS

Este proyecto muestra como funcionan los algoritmos de búsqueda BFS (Breadth-First Search) y DFS (Depth-First Search) utilizando Google Colab - Python.

El programa construye un grafo dirigido, ejecuta ambos algoritmos para recorrer los nodos y muestra el orden de visita.

Además, se genera una animación visual donde los nodos cambian de color a medida que son visitados.

El objetivo del proyecto es comprender cómo funcionan los recorridos de grafos y visualizar paso a paso el comportamiento de BFS y DFS.

Instalar las librerías necesarias antes de ejecutar el programa:

```
!pip install networkx
import networkx as nx
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
from IPython.display import HTML
from collections import deque
```

- networkx se utiliza para crear y manipular grafos.
- matplotlib permite dibujar el grafo.
- FuncAnimation genera la animación del recorrido.
- deque se usa como estructura de cola para BFS.

Creación del grafo

El grafo se representa mediante un diccionario donde cada nodo tiene una lista de vecinos.

```
grafo = {
"A": ["B", "C"],
"B": ["D", "E"],
"C": ["F", "G"],
"D": ["H"],
"E": [],
"F": [],
"G": [],
"H": []
}
```

Esto define las conexiones entre los nodos del grafo.

Construcción del grafo con NetworkX

Luego el diccionario se convierte en un grafo dirigido utilizando NetworkX.

```
G = nx.DiGraph()
```

```
for nodo in grafo:
    for vecino in grafo[nodo]:
        G.add_edge(nodo, vecino)
```

Cada conexión entre nodos se agrega como una arista dirigida.

Posición de los nodos

Para que el grafo se dibuje organizado, se define la posición de cada nodo.

```
pos = {
    "A": (0,0),
    "B": (-2,-1),
    "C": (2,-1),
    "D": (-3,-2),
    "E": (-1,-2),
    "F": (1,-2),
    "G": (3,-2),
    "H": (-3,-3)
}
```

Estas coordenadas indican dónde aparecerá cada nodo en el gráfico.

Implementación del algoritmo BFS

El algoritmo Breadth-First Search (BFS) recorre el grafo por niveles utilizando una cola.

```
def bfs(grafo, inicio):

    visitados = []
    cola = deque([inicio])

    while cola:

        nodo = cola.popleft()

        if nodo not in visitados:

            visitados.append(nodo)
```

```
        for vecino in grafo[nodo]:
            cola.append(vecino)

return visitados
```

- Se usa una cola (deque).
- Se visitan primero los nodos más cercanos al nodo inicial.
- Luego se visitan los nodos del siguiente nivel.

Implementación del algoritmo DFS

El algoritmo Depth-First Search (DFS) recorre el grafo en profundidad usando recursividad.

```
def dfs(grafo, nodo, visitados):
```

```
    if nodo not in visitados:
```

```
        visitados.append(nodo)
```

```
        for vecino in grafo[nodo]:
            dfs(grafo, vecino, visitados)
```

```
    return visitados
```

- Explora una rama completa antes de regresar.
- Utiliza recursión para recorrer el grafo.

Ejecución de los algoritmos

Se ejecutan ambos algoritmos comenzando desde el nodo A.

```
visitados_bfs = bfs(grafo, "A")
visitados_dfs = dfs(grafo, "A", [])
```

Luego se imprimen los resultados del recorrido.

```
print("Orden BFS:", visitados_bfs)
print("Orden DFS:", visitados_dfs)
```

Esto permite observar el orden en que cada algoritmo visita los nodos.

Animación del recorrido

Para visualizar el recorrido del grafo se utiliza una función que anima el proceso.

```
def animar_grafo(G, pos, visitados, titulo):
```

```
    fig, ax = plt.subplots()
```

```
    def actualizar(i):
```

```
        ax.clear()
```

```
        colores = [  
            "orange" if nodo in visitados[:i+1] else "lightblue"  
            for nodo in G.nodes()  
        ]
```

```
        ax.set_title(titulo + " - Paso " + str(i+1))
```

```
        nx.draw(G, pos, with_labels=True, node_color=colores, node_size=2000, ax=ax)
```

```
    animacion = FuncAnimation(fig, actualizar, frames=len(visitados), interval=1000)
```

```
    plt.close()
```

```
    return HTML(animacion.to_jshtml())
```

- Los nodos comienzan en azul claro.
- Cuando son visitados cambian a color naranja.
- La animación muestra el recorrido paso a paso.

Visualización final

Finalmente se muestran las animaciones de BFS y DFS.

```
display(animar_grafo(G, pos, visitados_bfs, "Recorrido BFS"))
```

```
display(animar_grafo(G, pos, visitados_dfs, "Recorrido DFS"))
```

Esto permite observar visualmente cómo cada algoritmo explora el grafo.